VALORIA VISION

Guide GitHub pas à pas

Phase 1 - Organisation, issues, sub-issues et documents

Version 1.0 - 19 juillet 2026
Public : François - Product Owner Valoria Vision
Périmètre : dépôt, documentation, backlog, roadmap, recette et releases

Référentiel officiel - à versionner dans le dépôt GitHub

Sommaire

1. Principes de gestion du projet

2. Structure du dossier docs/

3. Vocabulaire GitHub utile

4. Labels et milestones

5. Créer une Epic

6. Créer une sub-issue

7. Créer et qualifier un bug

8. Déposer le kit Phase 1 dans le dépôt

9. Mettre à jour ou remplacer un document

10. Créer et utiliser un Project GitHub

11. Cycle complet d'une évolution

12. Procédure de recette et fermeture

13. Procédure de release

14. Mise en place immédiate - Gestion des comptes

15. Checklists et dépannage

1. Principes de gestion du projet

> **Règle centrale**
> Aucune fonctionnalité n'est développée sans être identifiée dans le Product Backlog et tracée par une issue GitHub. Une issue n'est fermée qu'après recette réussie.

- GitHub Issues suit le travail concret : bugs, fonctionnalités, UX, technique et documentation.
- Les Epics regroupent un objectif large; les sub-issues représentent les éléments réellement développés et testés.
- Les milestones regroupent les issues d'une version ou d'une campagne corrective.
- Le Product Backlog Excel donne la vision complète et les dépendances; GitHub donne l'état opérationnel.
- Les documents du dossier docs/ sont versionnés avec le code afin de conserver leur historique.

2. Structure du dossier docs/

Dossier	Rôle
00_Gouvernance	Guide GitHub, règles de fonctionnement, modèles d'issues et checklists.
01_Product_Backlog	Classeur maître des fonctionnalités, bugs, priorités et versions.
02_Roadmap	Roadmap stratégique, planning indicatif et critères GO/NO GO.
03_Cahier_des_Charges	Cahier des charges vivant par module - construit en Phase 2.
04_ADR	Décisions d'architecture numérotées - construit en Phase 2.
05_Recette	Matrices de tests, suivi des anomalies et décisions GO/NO GO.
06_UX_Guidelines	Standards d'interface : tableaux, menus, boutons et lisibilité.
07_Communication	Documents de communication et suivi des plateformes.
08_Releases	Notes de versions, packages publiés et checklists de livraison.

> **Important**
> Git ne conserve pas les dossiers vides. Chaque dossier contient donc un fichier README.md qui explique son usage.

3. Vocabulaire GitHub utile

Terme	Définition Valoria Vision
Repository / dépôt	Espace contenant le code, les fichiers et leur historique.
Issue	Ticket utilisé pour planifier, discuter et suivre une tâche.
Epic	Issue parent décrivant un objectif large. GitHub n'impose pas ce nom : nous l'indiquons avec le préfixe [EPIC].
Sub-issue	Issue enfant rattachée à une issue parent. Sa fermeture alimente la progression du parent.
Label	Étiquette de type, priorité ou domaine.
Milestone	Regroupement d'issues associé à une version ou un jalon.
Project	Vue de pilotage synchronisée avec les issues et pull requests.
Commit	Enregistrement daté d'une modification de fichiers dans le dépôt.
Release	Publication officielle d'une version, accompagnée de notes et éventuellement d'un fichier ZIP.

4. Labels et milestones

4.1 Labels recommandés

Label	Usage
P0	Bloquant - empêche une sortie ou rend un parcours majeur inutilisable.
P1	Important - correction nécessaire ou contournement pénalisant.
P2	Amélioration - UX ou optimisation non bloquante.
P3	Idée - à conserver sans engagement immédiat.

bug	Anomalie reproductible.
feature	Nouvelle fonction métier.
ux	Ergonomie, lisibilité ou cohérence.
tech	Architecture, performance, migration ou maintenance.
documentation	Mise à jour des référentiels.
test	Recette, non-régression ou automatisation.
security	Sécurité et protection des données.

4.2 Milestones recommandées

- V0.6.6-R7 - Correctifs recette R6.1 : campagne actuelle, incluant R7.1, R7.2 et R7.3.
- V0.6.7 - Stabilisation et gestion des comptes : menus, modification, archivage, suppression et synchronisation.
- V0.6.8 - Security Foundation : audit, chiffrement, mot de passe maître et verrouillage.
- V0.7.0, V0.8.0, V0.9.0 et V1.0.0 : créer seulement lorsque la version précédente est suffisamment stabilisée.

> **Conseil pratique**
> Ne crée pas toutes les milestones futures immédiatement. La Roadmap Excel les conserve; GitHub peut rester centré sur la version actuelle et la suivante.

5. Créer une Epic

1 **Ouvrir le dépôt**
Sur la page principale du dépôt Valoria Vision, clique sur l'onglet Issues.

2 **Créer une issue**
Clique sur New issue. Si GitHub propose des modèles, choisis une issue vierge ou le modèle Epic.

3 **Saisir le titre**
Utilise le format : [EPIC] Gestion complète des comptes bancaires.

4 **Coller la description**
Utilise le texte préparé dans 00_Gouvernance/Issues_GitHub_Gestion_Comptes.md.

5 **Ajouter les labels**
Ajoute feature, ux et P1. Le label P1 représente la priorité globale; chaque sub-issue peut avoir sa propre priorité.

6 **Choisir la milestone**
Sélectionne V0.6.7 - Stabilisation et gestion des comptes.

7 **Créer**
Clique sur Submit new issue. Note le numéro de l'issue afin de le reporter dans le Product Backlog.

6. Créer une sub-issue

> **Disponibilité**
> L'interface GitHub permet d'ajouter des sub-issues depuis une issue parent. Le libellé exact du bouton peut légèrement varier; cherche la section Sub-issues dans l'Epic.

1 **Ouvrir l'Epic**
Ouvre l'issue [EPIC] Gestion complète des comptes bancaires.

2 **Trouver Sub-issues**
Dans la zone de relation ou dans le menu de l'issue, clique sur Create sub-issue ou Add sub-issue.

3 **Créer ou rattacher**
Choisis Create new issue pour une nouvelle sub-issue; Add existing issue pour rattacher une issue déjà créée.

4 **Renseigner le ticket**
Colle le titre et la description préparés. Ajoute priorité, type et milestone.

5 **Vérifier le parent**
La sub-issue doit afficher l'Epic comme parent. L'Epic affiche ensuite sa progression.

6 **Répéter**
Crée les sub-issues dans l'ordre proposé : rafraîchissement, menu, modification, archivage, suppression, filtres, tests.

7. Créer et qualifier un bug

1 **Titre factuel**
Format : [BUG] Le module Rapprochements ne rafraîchit pas la liste des comptes après création.

2 **Contexte**
Indique la version, le module et la campagne de recette.

3 **Étapes de reproduction**
Décris les actions exactes, une par ligne.

4 **Résultat obtenu**
Décris ce qui se passe réellement, sans interprétation.

5 **Résultat attendu**
Décris le comportement cible observable.

6 **Qualification**
Ajoute priorité P0/P1/P2, label bug et milestone actuelle.

7 **Preuves**
Ajoute captures ou fichiers de test sans données sensibles.

8 **Critère de fermeture**
Écris le test qui permettra de fermer l'issue après correction.

Cas détecté le 19/07/2026
Le compte est bien enregistré en base : il apparaît après redémarrage. L'anomalie est donc un défaut de rafraîchissement de l'interface, classé P1.

8. Déposer le kit Phase 1 dans le dépôt

Méthode conseillée pour débuter
Utilise le navigateur GitHub. Le kit ZIP contient un dossier docs/ prêt à déposer. GitHub enregistre chaque dépôt dans un commit.

1 **Décompresser le ZIP**
Dans Windows, clic droit sur ValoriaVision_Phase1_Referentiel.zip puis Extraire tout.

2 **Ouvrir le dépôt GitHub**
Place-toi à la racine du dépôt, là où apparaissent les dossiers du code.

3 **Choisir Add file**
Clique sur Add file puis Upload files.

4 **Déposer le dossier docs**
Glisse le dossier docs depuis l'Explorateur Windows dans la zone de dépôt. Si le navigateur refuse le dossier, ouvre docs et dépose son contenu en conservant les sous-dossiers.

5 **Contrôler la liste**
Vérifie la présence des dossiers 00_Gouvernance à 08_Releases et des fichiers Excel, Word, PDF et Markdown.

6 **Saisir le message de commit**
Écris : docs: ajoute le référentiel de pilotage Phase 1.

7 **Valider**
Choisis Commit directly to the main branch uniquement si tu es seul sur le dépôt et que tu acceptes l'ajout. Clique sur Commit changes.

8 **Vérifier**
Retourne à la racine, ouvre docs/README.md puis les fichiers pour confirmer leur présence.

Sécurité
Ne dépose jamais de mot de passe, token, clé API, base bancaire réelle ou sauvegarde contenant des données personnelles dans GitHub.

9. Mettre à jour ou remplacer un document

1. **Télécharger le document courant**
   Depuis GitHub, ouvre le fichier puis utilise Download raw file si tu as besoin d'une copie locale.

2. **Modifier la copie officielle**
   Travaille sur le fichier portant le même nom. Évite de créer des variantes final-final-2.

3. **Ouvrir le bon dossier**
   Dans GitHub, navigue jusqu'au dossier exact du document.

4. **Upload files**
   Dépose le fichier portant le même nom. GitHub proposera de remplacer la version existante.

5. **Message de commit**
   Utilise : docs: met à jour le Product Backlog après recette R7.1.

6. **Vérifier l'historique**
   Ouvre le fichier puis History pour voir les versions successives et revenir à une version antérieure si nécessaire.

Règle de nommage
Le nom du fichier reste stable; la date et la version du document sont indiquées dans son contenu et dans le message de commit.

10. Créer et utiliser un Project GitHub

1. **Créer le Project**
   Depuis ton profil ou l'onglet Projects, clique sur New project puis choisis une vue Table ou Board.

2. **Nommer**
   Valoria Vision - Pilotage produit.

3. **Ajouter les statuts**
   Backlog, Prêt, En cours, À retester, Terminé.

4. **Ajouter les champs**
   Priorité, Version cible, Type, Parent issue et Sub-issue progress si disponibles.

5. **Ajouter les issues**
   Ajoute les issues du dépôt. Les modifications de titre ou fermeture restent synchronisées.

6. **Créer des vues**
   Vue Sprint R7, Vue Gestion des comptes, Vue P0/P1, Vue Roadmap.

7 **Lier au dépôt**
Dans l'onglet Projects du dépôt, utilise Link a project pour le rendre accessible depuis le dépôt.

11. Cycle complet d'une évolution

Étape	Résultat attendu
1. Idée	Décrire le besoin et son bénéfice.
2. Analyse	Qualifier bug, feature, UX, tech ou documentation.
3. Backlog	Créer ou mettre à jour la ligne avec ID, priorité, version et critère.
4. GitHub	Créer l'issue et la rattacher à l'Epic et la milestone.
5. Cahier des charges	Documenter la règle métier - Phase 2.
6. Développement	Livrer une version installable identifiée.
7. Recette	Exécuter les tests liés et enregistrer les preuves.
8. Validation	Fermer l'issue uniquement si le test est OK.
9. Documentation	Mettre à jour backlog, roadmap, ADR et notes de version.
10. Release	Publier le package validé et les notes de version.

12. Procédure de recette et fermeture

1 **Passer le test**
Suivre la matrice sans modifier les préconditions en cours de test.

2 **Noter le résultat**
OK, KO ou OK avec réserve UX; ajouter commentaire, testeur et date.

3 **Mettre à jour l'issue**
Ajouter le résultat détaillé et les réserves éventuelles.

4 **Décider**
OK fonctionnel : fermer le bug; réserve UX : créer ou lier une issue UX distincte.

5 **Mettre à jour le backlog**
Statut Validé ou À retester; renseigner le numéro GitHub manquant.

6 **Mettre à jour le Sprint**
Ajouter un commentaire de synthèse dans l'issue Sprint R7.1.

13. Procédure de release

- Toutes les issues P0 sont fermées ou requalifiées avec justification.
- La matrice de recette est complète et la décision GO/NO GO est enregistrée.
- Le Product Backlog et la Roadmap sont à jour.
- Les migrations et sauvegardes ont été testées.
- Le package installable porte le bon numéro de version dans le titre et le pied de page.
- Les notes de version décrivent corrections, limitations et procédure de retour arrière.
- La Release GitHub contient le ZIP validé; aucune base réelle ni secret n'est joint.

14. Mise en place immédiate - Gestion des comptes

Ordre recommandé
Crée d'abord l'Epic, puis l'anomalie de rafraîchissement P1, puis les fonctionnalités de modification, archivage et suppression. Cela sépare la correction immédiate de l'évolution V0.6.7.

Ordre	Action	Réglage
1	Créer [EPIC] Gestion complète des comptes bancaires	Milestone V0.6.7; labels feature, ux, P1.
2	Créer [BUG] Rafraîchir la liste des comptes après création	Milestone R7 actuelle ou R7.2; labels bug, P1.
3	Créer les sub-issues V0.6.7	Menu contextuel, modification, archivage, suppression, filtres et

		tests transverses.
4	Reporter les numéros	Ouvrir Product Backlog, colonne Issue GitHub, remplacer À créer par #numéro.
5	Déposer le Product Backlog mis à jour	Commit : docs: renseigne les issues GitHub de la gestion des comptes.

15. Checklists et dépannage

15.1 Checklist après une décision produit

- Ligne Product Backlog créée ou mise à jour.
- Issue ou sub-issue créée.
- Priorité et version cible attribuées.
- Critères d'acceptation écrits.
- Tests prévus.
- Cahier des charges/ADR à mettre à jour si nécessaire.

15.2 Checklist après une correction

- Version installable identifiée.
- Test rejoué.
- Matrice mise à jour.
- Issue commentée puis fermée si OK.
- Réserve UX séparée du bug fonctionnel.
- Backlog et notes de version mis à jour.

15.3 Problèmes fréquents

Problème	Solution
Je ne vois pas Sub-issues	Actualise la page, vérifie que tu es dans une issue et cherche la zone Relations/Sub-issues. À défaut, crée les issues puis rattache-les depuis l'Epic.
GitHub refuse un dossier vide	Ajoute un README.md; le kit en contient déjà un dans chaque dossier.
Un fichier Excel ne s'affiche pas	GitHub stocke le fichier mais ne prévisualise pas toujours les fonctions avancées. Télécharge-le pour l'ouvrir dans Excel.
J'ai créé une issue au mauvais endroit	Modifie la milestone, les labels ou le parent; il n'est généralement pas nécessaire de recréer le ticket.
Je crains d'écraser un document	L'historique GitHub conserve les commits. Vérifie le nom, utilise un message de commit clair et contrôle le fichier après dépôt.

Le bouton n’a pas exactement le même nom	L’interface GitHub évolue. Cherche l’action équivalente dans Add file, Issues, Projects, Milestones ou le menu ⋯.

Références officielles GitHub

- Managing files in a repository
- Creating an issue
- Using labels and milestones
- Creating a project
- Parent issue and sub-issue progress fields
- Adding a project to a repository